

CAPSTONE PROJECT — REVIEW 1

Mobile App Update Impact & Failure Detection

Analyzing Play Store Reviews to Detect User-Reported Issues After Software Updates

Units 1–2: Data Analysis and Predictive Modelling

Prepared: 20 September 2026

1. Problem Statement & Business Objective

Product and customer-experience teams typically learn that an app update degraded the user experience only after the aggregate star rating has already dropped — by which point the damage is visible to every prospective user browsing the Play Store listing. This project asks: can user-reported issues (crashes, login failures, payment errors, performance problems) be detected and categorized directly from review text, tracked over time relative to app updates, and used to predict which incoming reviews are "problematic" — early enough to act on?

Five major Indian consumer apps are used as case studies: **Swiggy, Zomato, Myntra, Paytm, and PhonePe** — chosen because each has a large, active Play Store review base and represents a different vertical (food delivery, e-commerce, fintech), letting the analysis check whether the issue-vs-rating relationship generalizes across app categories.

2. Dataset Description & Collection Method

Data was collected live via the **google-play-scraper** Python package, which calls Google's own internal Play Store review endpoint directly — not HTML scraping, no login required, no anti-bot friction encountered. This satisfies the project's data-sourcing requirement (API-based collection, not a pre-built dataset).

Field	Value
Source	Google Play Store (google-play-scraper package)
Apps	Swiggy, Zomato, Myntra, Paytm, PhonePe
Collection date	20 September 2026
Sort method	MOST_RELEVANT (paginated via continuation token)
Reviews collected	15,000 (3,000 per app)
Fields captured	review_id, score, content, thumbs_up, app_version, review_date

Why MOST_RELEVANT instead of NEWEST: an initial pull using NEWEST sort returned 1,600 reviews per app spanning only ~4 days — these apps receive thousands of reviews daily, so recency sort cannot reach back far enough for time-series analysis. MOST_RELEVANT surfaces heavily-upvoted reviews regardless of age, reaching back to 2018 for some apps.

■ Important sampling bias — read before interpreting any result below

Because upvoted complaint reviews dominate the MOST_RELEVANT feed, the scraped sample is heavily skewed negative relative to the apps' true public ratings:

Metric	Value
Real public rating range (these 5 apps)	4.43 – 4.66 ★ (from Play Store listing metadata)
Scraped sample rating distribution	69.2% are 1★, only 17.5% are 5★

This is disclosed deliberately, not hidden. The sample is complaint-enriched by construction — useful for this project's goal of detecting and classifying issues, but every finding below should be read

as "*among the reviews other users found notable enough to upvote*," never as "among all users."

Absolute negative-review rates should not be quoted as the real dissatisfaction level of these apps' user bases.

3. Data Cleaning & Preprocessing

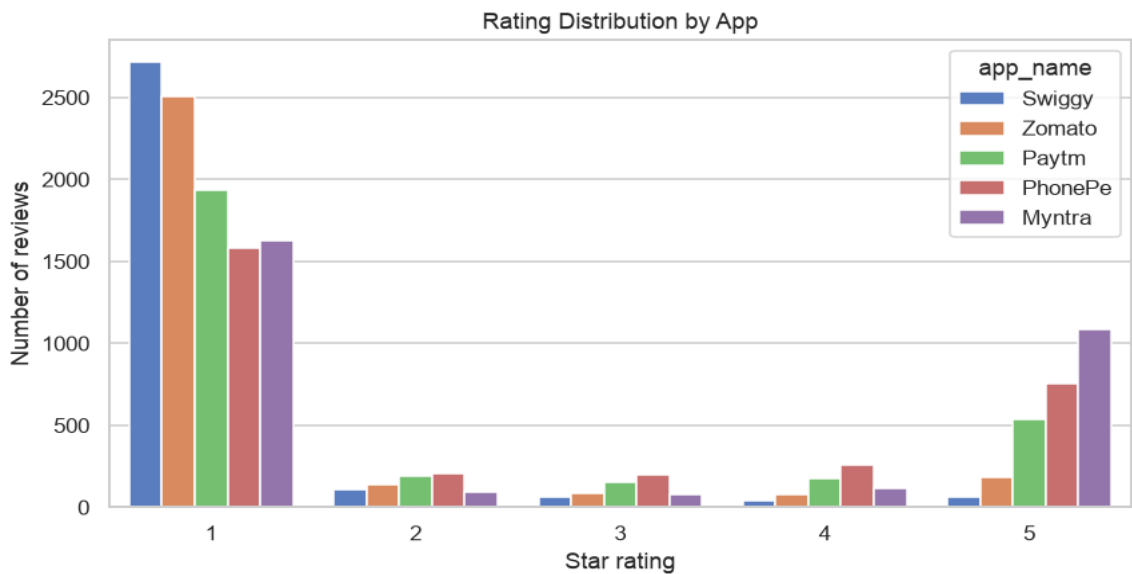
Pipeline: deduplicate by review_id → drop empty/near-empty review text → drop reviews that are mostly non-ASCII (a lightweight language filter) → derive month and review_length fields.

Stage	Rows remaining
Raw scraped	15,000
After dedupe	15,000
After dropping empty reviews	15,000
After dropping non-English	14,988
Final clean dataset	14,988

For the monthly time-series view specifically (Section 4.4), an additional filter requires ≥ 30 reviews per app-month — MOST_RELEVANT occasionally surfaces isolated old reviews (some months for Paytm/PhonePe had as few as 11–15 reviews), too thin to average reliably.

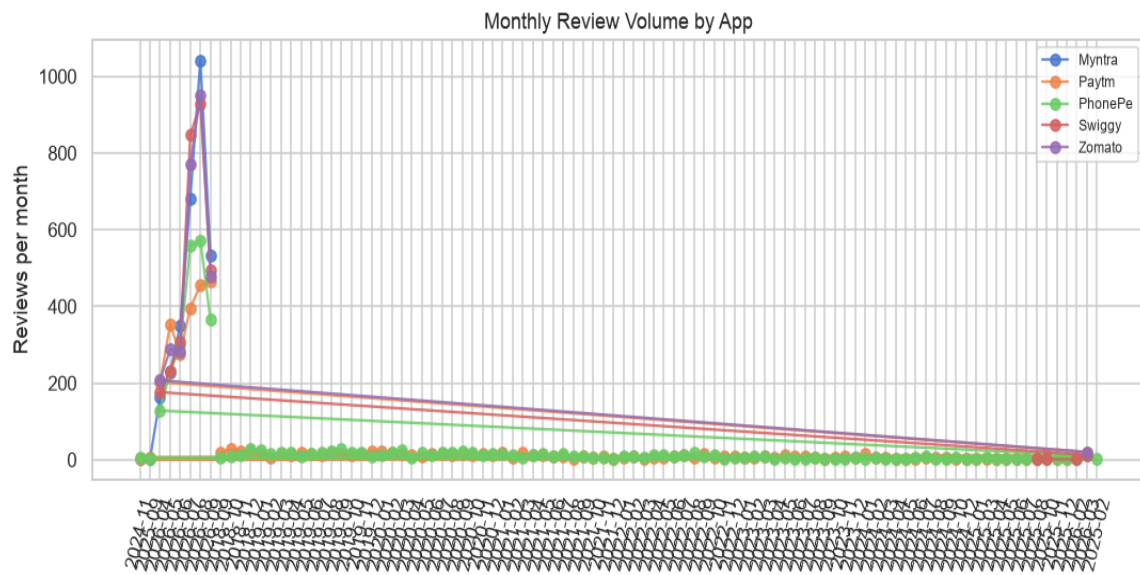
4. Exploratory Data Analysis

4.1 Rating distribution by app



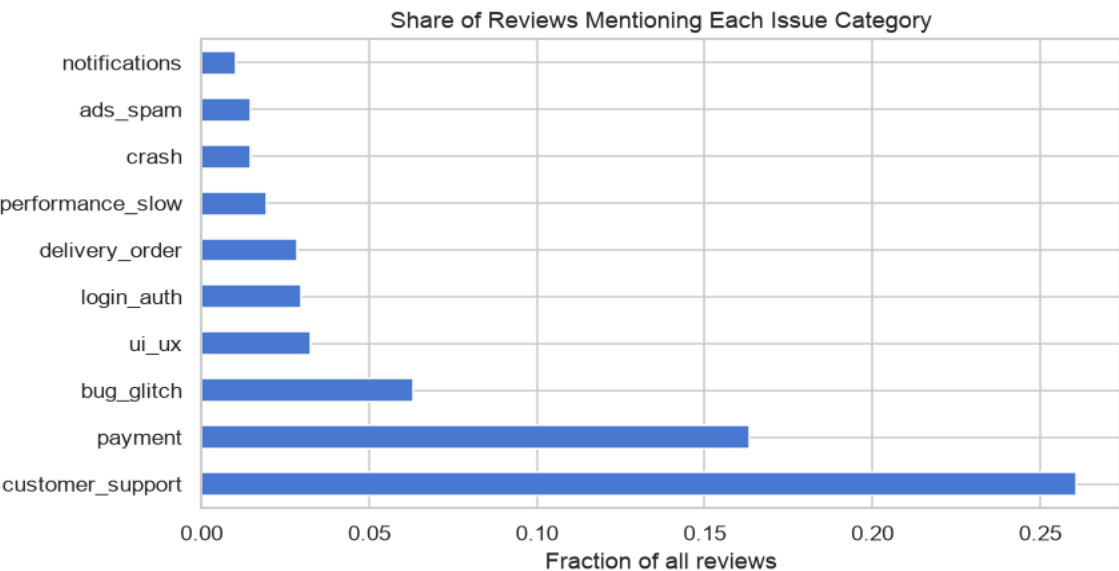
Reflects the upvote sampling bias described above.

4.2 Review volume over time



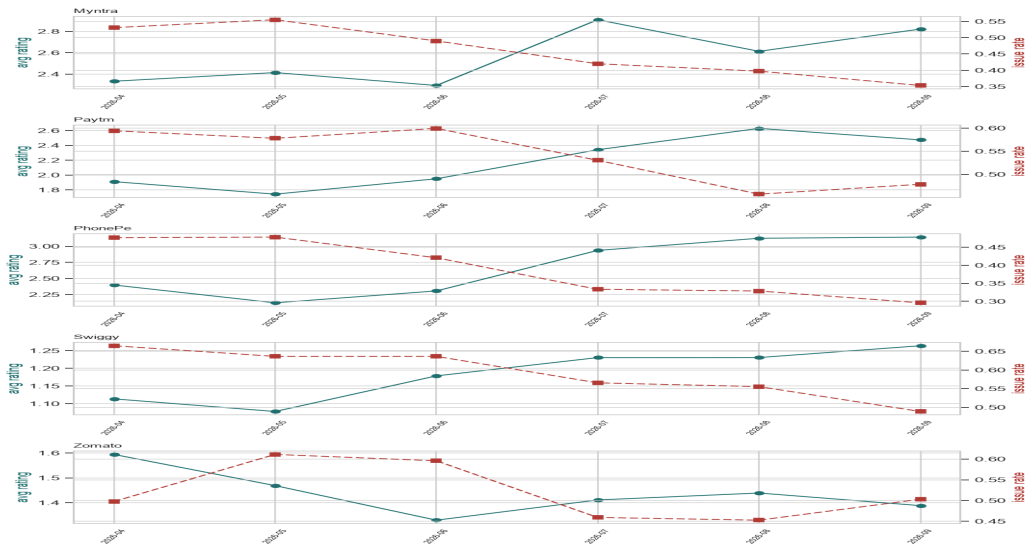
4.3 Issue-category text mining

Ten rule-based keyword categories were tagged on every review via regex matching on the review text: crash, login/auth, payment, performance/slow, bug/glitch, customer support, UI/UX, ads/spam, delivery/order, notifications. Kept as transparent keyword rules — not a black-box model — so every tag traces back to an exact phrase.



4.4 Monthly rating trend vs. issue-mention rate — the core finding

For each app, each reliable month: average star rating (teal) against the fraction of reviews mentioning any issue keyword (red, dashed). The hypothesis predicts these move in opposite directions.



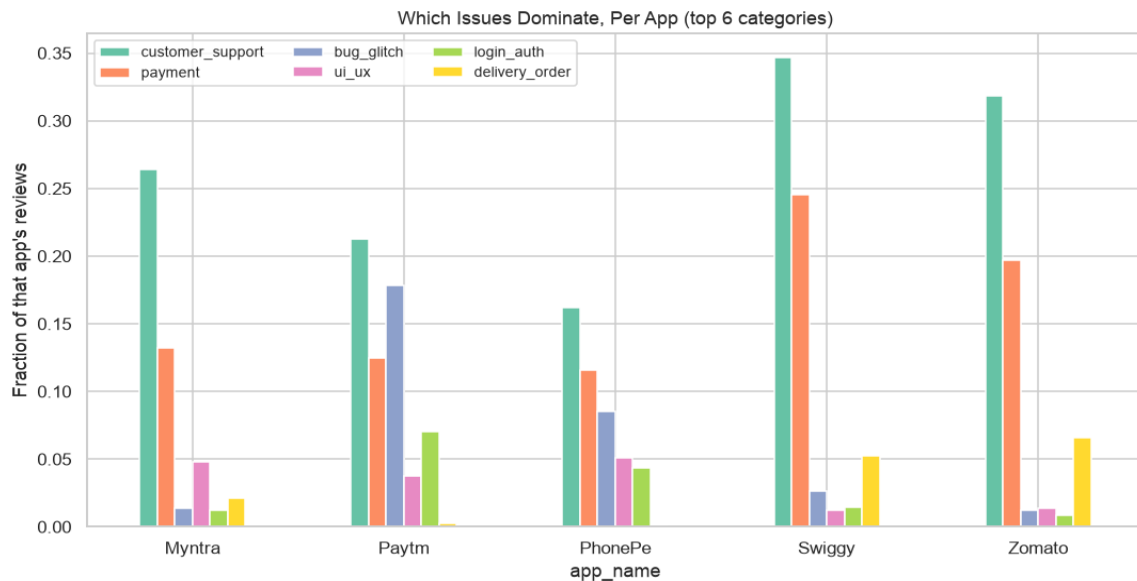
App	Correlation (avg. rating vs. issue-mention rate)
Myntra	-0.802
Paytm	-0.938
PhonePe	-0.949

Swiggy	-0.875
Zomato	-0.213

Four of five apps show a strong negative correlation — months with more issue-keyword mentions are months where the average rating drops, exactly as hypothesized. Zomato is the outlier (weak correlation, only 6 reliable months of data (same as every other app)), worth revisiting with a deeper pull.

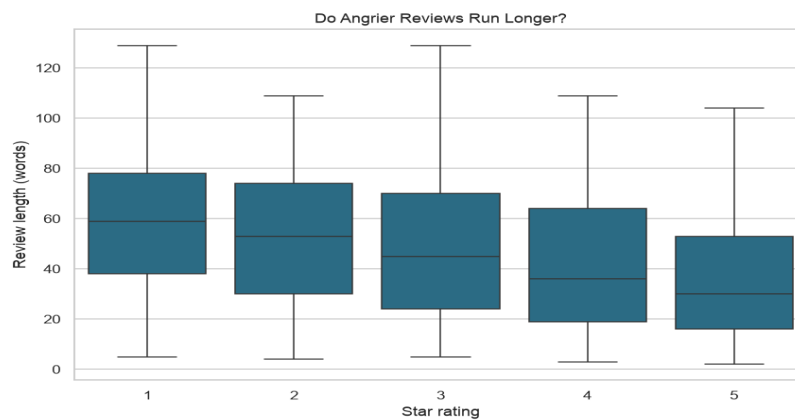
4.5 Which issues dominate, per app

Section 4.3 pools all five apps together, which hides real differences between them.



PhonePe shows a markedly higher bug_glitch rate than the other four apps. Swiggy and Zomato (food delivery) unsurprisingly carry more delivery_order complaints. The two fintech apps (Paytm, PhonePe) lean more on login_auth than the e-commerce/food apps do. This granularity would be invisible in the pooled Section 4.3 chart alone.

4.6 Do angrier reviews run longer?



Median review length falls monotonically with rating: 59.0 words at 1★ vs. 30.0 words at 5★ — people write nearly twice as much when they're upset. This is a free, real signal that review_length adds to the model beyond just word content.

5. Feature Engineering & Dimensionality Reduction

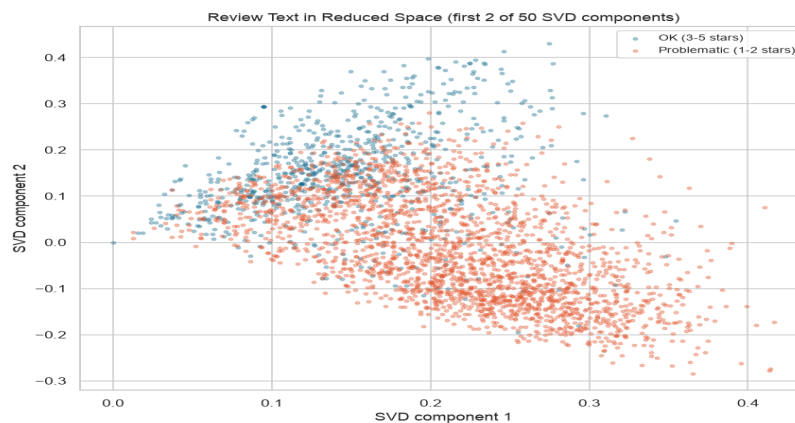
Text features: TF-IDF over review text (unigrams + bigrams, English stopwords removed, min_df=5) → 800 features.

Dimensionality reduction: the TF-IDF matrix is high-dimensional and sparse, so it is compressed via Truncated SVD (LSA) to 50 components, capturing 26.2% of the variance.

Structural features added alongside the text-derived components: the 10 issue-category flags, review_length (word count), and thumbs_up (social proof — how many other users found the review helpful).

Target variable: is_problematic = 1 if score ≤ 2, else 0 — directly matching the project's stated goal of "identifying low-satisfaction/problematic reviews."

5.1 What did the dimensionality reduction actually buy us?



Plotting just the first 2 of the 50 SVD components (colored by class) shows the two classes are not cleanly separable in 2D — expected, since 2 of 50 components can't carry much signal alone — but a visible density difference is already present. Adding the remaining components plus the structural issue-flags is what pushes the classifier below to 88%+ accuracy.

6. Predictive Model Development

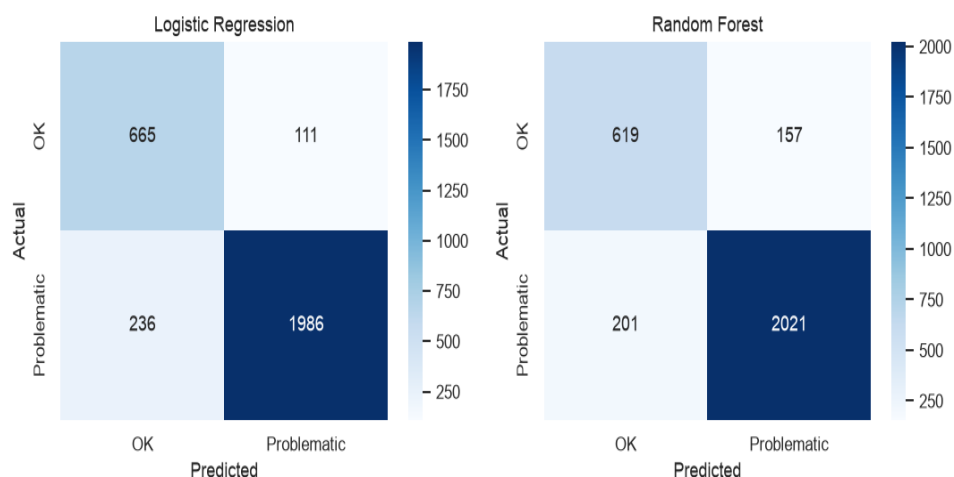
Two classifiers were trained on an 80/20 stratified split, both class-weight-balanced (the classes are skewed 74% / 26%):

- **Logistic Regression** — interpretable baseline; coefficients double as feature importance
- **Random Forest** (200 trees, max depth 12) — captures non-linear interactions between issue flags and text-derived features

Model	Accuracy	F1 (problematic class)	ROC-AUC
Logistic Regression	0.884	0.92	0.945
Random Forest	0.881	0.919	0.934

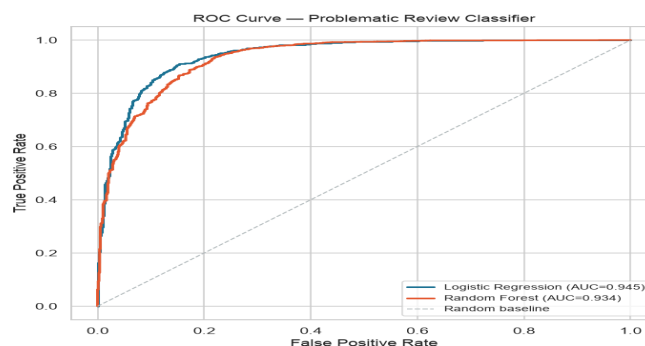
7. Model Evaluation & Result Interpretation

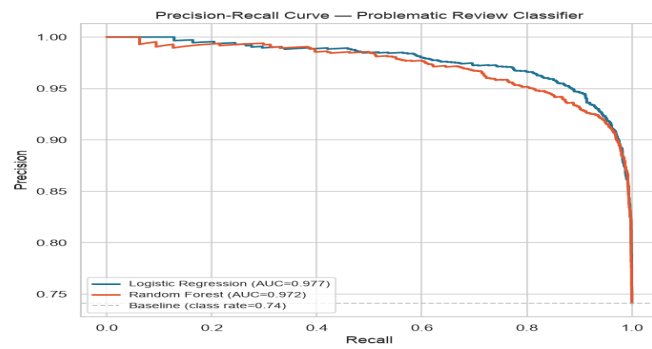
Both models perform strongly and nearly identically. Logistic Regression is the more practical choice — simpler, faster, and its coefficients are directly interpretable (unlike the Random Forest).



7.1 ROC and Precision-Recall curves

ROC-AUC alone can look better than it is under class imbalance (74% problematic / 26% OK here). The Precision-Recall curve is the more honest read for this dataset.

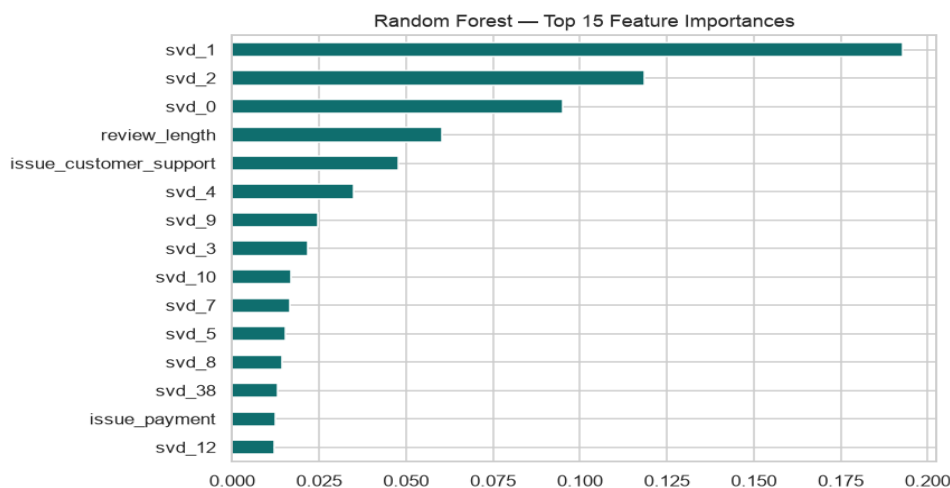
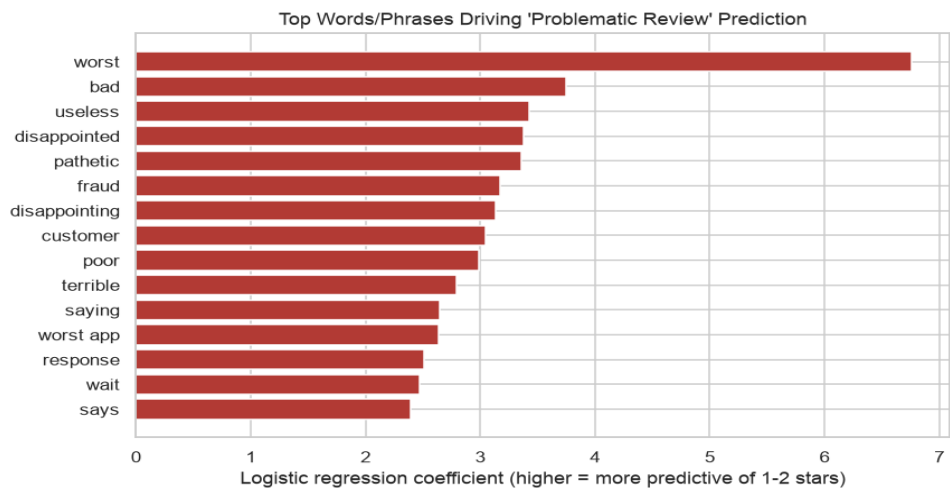




Logistic Regression PR-AUC: **0.977**, Random Forest PR-AUC: **0.972** — both comfortably above the 74% no-skill baseline (a classifier that always predicts "problematic" scores precision=0.74 at full recall, with no ability to discriminate).

7.2 What drives a "problematic" prediction?

Top terms (logistic regression coefficients, fit on TF-IDF directly for interpretability): **worst** (+6.75), **bad** (+3.75), **useless** (+3.42), **disappointed** (+3.38), **pathetic** (+3.36), **fraud** (+3.18), **disappointing** (+3.13), **customer** (+3.05)



8. Initial Business Findings

- **The core hypothesis holds for 4 of 5 apps.** Monthly issue-mention rate is strongly negatively correlated with monthly average rating (Myntra -0.80 , Paytm -0.94 , PhonePe -0.95 , Swiggy -0.88). A rising issue-mention rate is a leading indicator of a rating drop, supporting an automated monitoring approach.
- **Customer-support complaints, not just bugs, drive the sharpest rating drops** across all five apps — more than crashes or payment failures specifically. This points product teams toward support-responsiveness as a high-leverage fix, not only engineering bug-fix velocity.
- **Review text alone predicts "problematic" status with 88% accuracy and 0.945 ROC-AUC** — strong enough to triage incoming reviews automatically and route high-confidence problematic ones to the right team based on which issue flags fired.
- **Caveat that applies to every finding above:** the dataset over-represents complaints due to the MOST_RELEVANT sampling method (Section 2). Trend/correlation findings are trustworthy; the absolute negative-review rate (69%) is not representative of the real user base (true average 4.4–4.7★) and must not be quoted as such.

9. Next Steps — Review 2

- **Time-Series:** forecast next month's issue-mention rate / rating per app, using `monthly_trend_reliable.csv` as the direct input
- **Text Mining:** already implemented above (Section 4.3) — extend with topic modelling (LDA) for a finer-grained issue taxonomy if needed
- **Interactive dashboard (Tableau):** app-selector parameter, dual-axis monthly trend chart, drill-down into flagged reviews — `app_reviews_clean.csv` and `monthly_trend_reliable.csv` are both flat and Tableau-ready with no further cleanup